

CLAFS — Campus Lost-and-Found System

API Documentation / Notes

Phase 3 submission · ITS122P — AM2 · Group 4

Samuela Ysebelle Adame · Kervin Del Rosario · Mariah Kate Guiang · Kendrick Sebastian
Mapua University

Contents

1. Overview	2
2. CLAFS JSON API (internal)	2
2.1 API purpose	2
2.2 API endpoints	2
2.3 Data retrieved from the API	4
2.4 How the API is integrated into the website	4
3. Nager.Date Public Holiday API (external)	5
3.1 API used	5
3.2 API purpose	5
3.3 API endpoint	6
3.4 Data retrieved	6
3.5 How it is integrated	6
4. Where to see the API working	6

1. Overview

CLAFS uses two APIs. The first is the CLAFS JSON API — a set of PHP endpoints in the `api/` folder that the website's own JavaScript calls with `fetch()` to read and write lost-and-found records without reloading the page. The second is an external, third-party API — Nager.Date public holidays — which the browser calls directly to show when the Lost & Found office is closed.

API	Type	Base URL	Format	Auth
CLAFS JSON API	Internal (own back end)	<code>`/api/`</code> e.g. <code>http://localhost:8000/api/</code>	JSON	PHP session cookie (log in first)
Nager.Date Public Holiday API	External / third-party	<code>`https://date.nager.at/api/v3/`</code>	JSON	None (free, no key)

Files involved: `public/js/api.js` (`fetch()` wrappers for every call), `public/js/app.js` (form handlers, live search, holiday widget), `api/*.php` (server endpoints), `config/db_connect.php` (shared validation, JSON helpers, database access).

2. CLAFS JSON API (internal)

2.1 API purpose

The CLAFS JSON API is the bridge between the browser and the MySQL database. Every action that changes data — filing a lost report, logging a found item, submitting or reviewing a claim, changing a status, managing users — is sent to one of these endpoints with `fetch()` and the response is used to update the page in place. The public found-items search also reads from it, so results appear as the user types.

Rules that protect the data (who may see or change what) are enforced on the server in each endpoint; the JavaScript validation in `app.js` is only a convenience for the user.

2.2 API endpoints

Endpoint	Method	Who	Purpose
<code>`api/get_items.php`</code>	GET	Anyone (found) / logged in (lost)	List found items or lost reports with keyword, category, status, date and sort filters
<code>`api/add_item.php`</code>	POST (multipart)	Logged in (lost) / staff (found)	Create a lost report or found item, with an optional photo upload
<code>`api/update_item.php`</code>	POST (multipart)	Owner (lost) / staff (found)	Edit an existing report or item; optionally replace the photo
<code>`api/update_status.php`</code>	POST	Staff; owner may close own report	Change a status (stored/returned/disposed, open/matched/closed). “Returned” also closes the linked report and rejects other pending claims

Endpoint	Method	Who	Purpose
`api/claims.php`	POST	Logged in / staff	action=create (submit a claim), review (approve/reject), withdraw
`api/update_user.php`	POST	Admin	Change a user's role or activate/deactivate the account

Common response shape. Every endpoint returns JSON with an **ok** flag. Errors use the matching HTTP status code (401 not logged in, 403 not allowed, 404 not found, 409 conflict, 422 validation) and a message; validation errors also include an **errors** object keyed by field name so the form can highlight the exact field.

```
// success                                // validation failure (HTTP 422)
{ "ok": true, ... }                       { "ok": false,
                                         "error": "Please fix the highlighted fields.",
                                         "errors": { "description": "Must be at least 20
characters." } }
```

GET /api/get_items.php

Query parameters: **type**=found|lost, **q** (keyword), **category**, **status**, **from**, **to** (dates), **sort**=newest|oldest, **limit** (1–100).

```
GET /api/get_items.php?type=found&q=key&sort=newest&limit=100

{ "ok": true, "type": "found", "count": 1,
  "items": [ { "item_id": 5, "item_name": "Silver keychain with 3 keys", "category": "Keys",
    "description": "Keychain with three keys found near the motorcycle parking
area.",
    "location_found": "Parking Area", "date_found": "2026-09-11",
    "image_url": null, "status": "stored", "created_at": "2026-09-11 08:50:00" }
  ] }
```

POST /api/add_item.php and /api/update_item.php

Sent as **multipart/form-data** (a **FormData** built from the form) so the photo file can be included. Fields: **type**, **item_name**, **category**, **description**, **date_lost** + **location_lost** (lost) or **date_found** + **location_found** + **storage_location** + **private_details** (found), optional **photo**. **update_item.php** also takes **id**.

```
POST /api/add_item.php          (type=lost, item_name=..., photo=<file>)

HTTP 201  { "ok": true, "type": "lost", "id": 6, "url": "/view_item.php?type=lost&id=6",
  "item": { ... } }
```

POST /api/update_status.php

```
POST /api/update_status.php    Content-Type: application/json
```

```
{ "type": "found", "id": 2, "status": "returned" }

{ "ok": true, "type": "found", "id": 2, "previous": "stored", "status": "returned",
  "message": "Item #2 is now "Returned".", "rejected_claims": [3] }
```

For lost reports, `status=matched` also accepts `item_id` — the found item staff matched the report to — which is stored in `lost_reports.matched_item_id` and shown to the owner.

POST /api/claims.php

```
{ "action": "create",      "item_id": 4, "proof_description": "...", "report_id": 1,
  "confirm_truth": 1 }
{ "action": "review",     "claim_id": 4, "decision": "approve", "review_note": "Bring a valid
  ID ..." }
{ "action": "withdraw",   "claim_id": 5 }

{ "ok": true, "action": "review", "claim_id": 4, "status": "approved",
  "message": "Claim #4 approved. The claimant can now see your note.", "claim": { ... } }
```

POST /api/update_user.php

```
{ "user_id": 4, "role": "staff" }      or      { "user_id": 4, "is_active": 0 }

{ "ok": true, "user_id": 4, "role": "staff", "is_active": 1, "message": "Bea Lim: role set
to Security & Maintenance." }
```

2.3 Data retrieved from the API

Endpoint	Data returned	Used for
get_items.php	Array of found items (item_id, item_name, category, description, location_found, date_found, image_url, status) or lost reports. Staff also receive storage_location and private_details.	Rendering the item cards on the Found Items page as the user searches
add_item.php / update_item.php	The saved record plus its id and page URL	Redirecting to the new/updated item page
update_status.php	New and previous status, a message, ids of claims auto-rejected	Updating every status badge on the page and showing a banner
claims.php	The claim row, its new status and a message	Replacing the form with a confirmation, revealing the hand-over step, removing a withdrawn claim
update_user.php	Updated role / active flag and a message	Updating the Active/Deactivated badge and flipping the button

2.4 How the API is integrated into the website

Client side. `public/js/api.js` defines a global `ClafsApi` object with one function per endpoint (`getItems`, `addItem`, `updateItem`, `updateStatus`, `createClaim`, `reviewClaim`, `withdrawClaim`, `updateUser`). Each one calls `fetch()`, sends JSON or FormData, parses the JSON reply, and rejects with an Error that carries the HTTP status and the field errors when the server says `ok:false`.

```
// public/js/api.js (excerpt)
return fetch(url, init).then(function (response) {
  return response.json().then(function (data) {
    if (!response.ok || data.ok === false) {
      var error = new Error(data.error); error.status = response.status; error.data =
data;
      throw error;
    }
    return data;
  });
});
```

Forms. A form opts in with `data-api="add_item"` (or `update_item`, `update_status`, `claim`, `review`, `withdraw`, `update_user`). `app.js` intercepts its submit event, runs client-side validation, calls the matching `ClafsApi` function, and then — instead of reloading — updates the page: badges with `data-status-for="found-2"` change text and colour, `data-done="replace"` swaps the form for a success alert, `data-done="remove"` deletes the card. Server-side validation errors (HTTP 422) are written under the exact field they belong to.

Live search. The search form on `browse.php` has `data-live-search`. Typing in the keyword box (debounced 300 ms) or changing a filter calls `ClafsApi.getItems()`, and the returned items are rendered into cards by JavaScript; the URL is updated with `history.replaceState` so the search can still be shared. Without JavaScript the same form submits normally and PHP renders the results.

Server side. Each `api/*.php` file requires `config/db_connect.php`, checks the session and role, validates input with the shared `validate_item_input()` helper, runs prepared PDO statements against MySQL, and replies with `json_response()` / `json_error()`. The endpoints accept ordinary form fields as well as JSON, so they also work without JavaScript.

3. Nager.Date Public Holiday API (external)

3.1 API used

Nager.Date — a free, open public-holiday web service (<https://date.nager.at>). No registration or API key is required and the API allows cross-origin requests, so the browser can call it directly.

3.2 API purpose

The Lost & Found office is closed on Philippine public holidays. The website fetches the official holiday list and shows the next closure date on the landing page (Office hours card) and in the Pickup details of every found item, so students know when they can actually collect an item.

3.3 API endpoint

```
GET https://date.nager.at/api/v3/PublicHolidays/{year}/PH
```

```
e.g. GET https://date.nager.at/api/v3/PublicHolidays/2026/PH
```

3.4 Data retrieved

A JSON array with one object per holiday, sorted by date. Fields used by CLAFS are date, name and localName.

```
[ { "date": "2026-10-31", "localName": "All Saints' Day Eve", "name": "All Saints' Day Eve",  
  "countryCode": "PH", "global": true, "types": ["Public"] },  
  { "date": "2026-11-01", "localName": "Undas", "name": "All Saints' Day", ... }, ... ]
```

3.5 How it is integrated

Elements marked `data-next-holiday` (index.php and view_item.php) are placeholders. On page load `app.js` calls `ClafsApi.getHolidays(year)`, which fetches the endpoint above with `fetch()`. The script keeps the holidays on or after today (falling back to next year's list when none are left), takes the first one, and writes "Next closure: Oct 31, 2026 — All Saints' Day Eve" into the placeholder. If the request fails (offline, service down) the placeholder shows "Holiday schedule unavailable right now." and the rest of the page is unaffected.

```
// public/js/app.js (excerpt)  
ClafsApi.getHolidays(year)  
  .then(function (holidays) {  
    var upcoming = holidays.filter(function (h) { return h.date >= todayIso; });  
    return upcoming.length ? upcoming : ClafsApi.getHolidays(year + 1);  
  })  
  .then(function (upcoming) { node.innerHTML = "Next closure: <strong>" +  
formatDate(upcoming[0].date) + "</strong> — " + upcoming[0].localName; })  
  .catch(function () { node.textContent = "Holiday schedule unavailable right now."; });
```

4. Where to see the API working

Page	What to try	API call made
Found Items (browse.php)	Type in the search box or change a filter	GET get_items.php — cards update without reload
Home page (index.php)	Look at the Office hours card	GET date.nager.at ... /PH — next closure appears
Report lost item (report.php)	Submit the form (try leaving fields empty first)	POST add_item.php — field errors / redirect to the new report
Item page (view_item.php)	Submit a claim as a student; approve it and mark returned as staff	POST claims.php, POST update_status.php — badges update in place

Page	What to try	API call made
Manage found items (browse.php?manage=1)	Change a status in the table	POST update_status.php
Users (index.php?tab=users, admin)	Change a role or deactivate an account	POST update_user.php

Open the browser's developer tools (F12 → Network tab) while doing any of the above to watch the fetch() requests and JSON responses.